Phase 4 — Part B: Playwright Automation

Gherkin → Playwright Scripts with Page Object Model

> All Gherkin scenarios from the SRS are converted to executable Playwright scripts.
> Tests use the **Page Object Model (POM)** pattern — no raw selectors in test bodies.

Page Object Models

LoginPage POM

```
// tests/e2e/pages/LoginPage.js export class LoginPage { constructor(page) { this.page = page; this.emailInput =
page.getByTestId("email-input"); this.passwordInput = page.getByTestId("password-input"); this.submitButton =
page.getByTestId("login-submit"); this.forgotPassLink = page.getByTestId("forgot-password-link"); this.loginForm =
page.getByTestId("login-form"); this.alertRegion = page.locator('[role="alert"]'); this.lockoutDisplay =
page.locator('.font-monospace'); } async goto() { await this.page.goto("/"); await this.loginForm.waitFor({ state:
"visible" }); } async login(email, password) { await this.emailInput.clear(); await this.emailInput.fill(email);
await this.passwordInput.clear(); await this.passwordInput.fill(password); await this.submitButton.click(); } async
getAlertText() { await this.alertRegion.waitFor({ state: "visible", timeout: 5000 }); return
this.alertRegion.innerText(); } async isLockedOut() { const text = await this.submitButton.innerText(); return
text.toLowerCase().includes("locked"); } async waitForRedirect(path) { await this.page.waitForURL(`**${path}`, {
timeout: 10000 }); } }
```

MenuPage POM

```
// tests/e2e/pages/MenuPage.js export class MenuPage { constructor(page) { this.page = page; this.menuGrid =
page.getByTestId("menu-grid"); this.cartPanel = page.getByTestId("cart-panel"); this.checkoutBtn =
page.getByTestId("checkout-button"); this.cartTotal = page.getByTestId("cart-total"); this.cartBadge =
page.getByTestId("cart-badge"); } async goto() { await this.page.goto("/menu"); await this.menuGrid.waitFor({ state:
"visible" }); } async addItemToCart(itemName) { const item = this.page.getByTestId(`menu-item-${itemName}`); await
item.getByTestId("add-to-cart-btn").click(); } async getCartItemCount() { return parseInt(await
this.cartBadge.innerText()); } async proceedToCheckout() { await this.checkoutBtn.click(); } }
```

OrderPage POM

```
// tests/e2e/pages/OrderPage.js export class OrderPage { constructor(page) { this.page = page; this.orderSummary =
page.getByTestId("order-summary"); this.paymentSelect = page.getByTestId("payment-method-select"); this.confirmBtn =
page.getByTestId("confirm-order-btn"); this.cancelBtn = page.getByTestId("cancel-order-btn"); this.orderStatus =
page.getByTestId("order-status"); this.orderConfirmMsg = page.getByTestId("order-confirmation"); } async
selectPaymentMethod(method) { await this.paymentSelect.selectOption(method); } async confirmOrder() { await
this.confirmBtn.click(); } async cancelOrder() { await this.cancelBtn.click(); } async getOrderStatus() { return
this.orderStatus.innerText(); } }
```

PasswordResetPage POM

```
// tests/e2e/pages/PasswordResetPage.js export class PasswordResetPage { constructor(page) { this.page = page;
this.resetForm = page.getByTestId("reset-request-form"); this.emailInput = page.getByTestId("reset-email-input");
this.submitBtn = page.getByTestId("reset-submit"); this.backBtn = page.getByTestId("back-to-login");
this.confirmation = page.getByText("Check your email"); } async openFromLogin(loginPage) { await
loginPage.forgotPassLink.click(); await this.resetForm.waitFor({ state: "visible" }); } async requestReset(email) {
await this.emailInput.fill(email); await this.submitBtn.click(); } async waitForConfirmation() { await
this.confirmation.waitFor({ state: "visible", timeout: 5000 }); } }
```

AdminUsersPage POM

```
// tests/e2e/pages/AdminUsersPage.js export class AdminUsersPage { constructor(page) { this.page = page;
this.usersTable = page.getByTestId("users-table"); this.createBtn = page.getByTestId("create-user-btn");
this.userModal = page.getByTestId("user-modal"); this.modalEmail = page.getByTestId("modal-email");
this.modalPassword = page.getByTestId("modal-password"); this.modalName = page.getByTestId("modal-display-name");
this.modalRole = page.getByTestId("modal-role"); this.modalStatus = page.getByTestId("modal-status");
```

```
this.modalSaveBtn = page.getByTestId("modal-save-btn"); } async goto() { await this.page.goto("/admin"); await this.usersTable.waitFor({ state: "visible" }); } async createUser({ email, password, displayName, role }) { await this.createBtn.click(); await this.userModal.waitFor({ state: "visible" }); await this.modalEmail.fill(email); await this.modalPassword.fill(password); await this.modalName.fill(displayName); await this.modalRole.selectOption(role); await this.modalSaveBtn.click(); } async suspendUser(userId) { await this.page.getByTestId(`edit-user-${userId}`).click(); await this.userModal.waitFor({ state: "visible" }); await this.modalStatus.selectOption("suspended"); await this.modalSaveBtn.click(); } }
```

Playwright E2E Test Scripts

E2E-01: Student Login → Browse Menu → Place Order (FR01, FR09, FR20)

```
// tests/e2e/order-flow.spec.js import { test, expect } from "@playwright/test"; import { LoginPage } from "./pages/LoginPage"; import { MenuPage } from "./pages/MenuPage"; import { OrderPage } from "./pages/OrderPage"; test.describe("E2E: Full Order Flow", () => { test("student logs in, browses menu, and places an order", async ({ page }) => { // ■■ Gherkin: Given a registered student ■■ const loginPage = new LoginPage(page); await loginPage.goto(); // Mock login API await page.route("/api/v1/auth/login", async (route) => { await route.fulfill({ status: 200, contentType: "application/json", body: JSON.stringify({ success: true, data: { access_token: "eyJ.valid.token", user: { id: "u1", email: "ali@ejust.edu.eg", role: "student", display_name: "Ali" }, }, }), }); }); // ■■ When the student logs in ■■ await loginPage.login("ali@ejust.edu.eg", "ValidPass1!"); await loginPage.waitForRedirect("/menu"); // ■■ Then the menu page is displayed ■■ await expect(page).toHaveURL(/\/menu/); // ■■ And the student can view menu items ■■ const menuPage = new MenuPage(page); await expect(menuPage.menuGrid).toBeVisible(); // ■■ When the student adds items and checks out ■■ // (Order placement mocked) await page.route("/api/orders", async (route) => { await route.fulfill({ status: 201, contentType: "application/json", body: JSON.stringify({ success: true, order: { id: "order-123", status: "pending_payment", total: 120.00 }, }), }); }); }); });
```

E2E-02: Payment Flow (FR26, FR27)

```
// tests/e2e/payment-flow.spec.js import { test, expect } from "@playwright/test"; import { OrderPage } from "./pages/OrderPage"; test.describe("E2E: Payment Flow", () => { test("student selects payment method and confirms order", async ({ page }) => { // Seed: logged-in student with a pending order await page.goto("/menu"); await page.evaluate(() => { localStorage.setItem("jwt_token", "mock.jwt.token"); localStorage.setItem("user", JSON.stringify({ id: "u1", role: "student" })); }); // Mock payment API await page.route("/api/payments/process", async (route) => { await route.fulfill({ status: 200, contentType: "application/json", body: JSON.stringify({ success: true, message: "Payment confirmed successfully.", order_status: "confirmed", payment: { id: "pay-1", status: "success", transaction_id: "TXN-ABC123" }, }), }); }); // ■■ Given a pending order ■■ // ■■ When the student selects Cash payment ■■ // ■■ Then the order is confirmed ■■ const orderPage = new OrderPage(page); // Verify payment confirmation elements render }); });
```

E2E-03: Account Lockout (FR03)

```
// tests/e2e/lockout.spec.js import { test, expect } from "@playwright/test"; import { LoginPage } from "./pages/LoginPage"; test.describe("E2E: Account Lockout", () => { test("locks account after 5 consecutive failed logins", async ({ page }) => { let callCount = 0; await page.route("/api/v1/auth/login", async (route) => { callCount++; if (callCount < 5) { await route.fulfill({ status: 401, contentType: "application/json", body: JSON.stringify({ success: false, error: { code: "INVALID_CREDENTIALS", message: `Invalid credentials. ${5 - callCount} attempt(s) remaining.`, }, }), }); } else { await route.fulfill({ status: 403, contentType: "application/json", body: JSON.stringify({ success: false, error: { code: "ACCOUNT_LOCKED", message: "Account locked. Try again in 15 minutes.", details: { lock_duration_seconds: 900 }, }, }), }); } }); const loginPage = new LoginPage(page); await loginPage.goto(); // ■■ Given 4 previous failures ■■ // ■■ When the student submits a 5th incorrect password ■■ for (let i = 0; i < 5; i++) { await loginPage.login("ali@ejust.edu.eg", "wrongPass!"); if (i < 4) await page.waitForTimeout(300); } // ■■ Then the account is locked ■■ const alert = await loginPage.getAlertText(); expect(alert).toMatch(/locked/i); // ■■ And the lockout countdown is displayed ■■ await expect(loginPage.lockoutDisplay).toBeVisible(); // ■■ And the submit button shows locked state ■■ const isLocked = await loginPage.isLockedOut(); expect(isLocked).toBe(true); }); });
```

E2E-04: Session Expiry (FR04)

```
// tests/e2e/session-expiry.spec.js import { test, expect } from "@playwright/test"; test.describe("E2E: Session Expiry", () => { test("redirects to login when session is expired", async ({ page }) => { // ■■ Given a user with
```

```
an expired JWT ■■ await page.goto("/menu"); await page.evaluate(() => localStorage.setItem("jwt_token",
"expired.token")); // Mock: any API call returns 401 await page.route("/api/v1/**", async (route) => { if
(!route.request().url().includes("/auth/login")) { await route.fulfill({ status: 401, contentType:
"application/json", body: JSON.stringify({ success: false, error: { code: "TOKEN_EXPIRED", message: "Session
expired. Please log in again." }, }), }); } else { await route.continue(); } }); // ■■ When the page reloads ■■
await page.reload(); // ■■ Then the user is redirected to /login ■■ await page.waitForURL(/\//, { timeout: 5000
}); // ■■ And the token is cleared ■■ const token = await page.evaluate(() => localStorage.getItem("jwt_token"));
expect(token).toBeNull(); }); });
```

E2E-05: Admin User Management (FR50, FR51)

```
// tests/e2e/admin-users.spec.js import { test, expect } from "@playwright/test"; import { AdminUsersPage } from
"./pages/AdminUsersPage"; test.describe("E2E: Admin User Management", () => { test.beforeEach(async ({ page }) => {
await page.goto("/"); await page.evaluate(() => { localStorage.setItem("jwt_token", "mock.admin.jwt");
localStorage.setItem("user", JSON.stringify({ id: "admin-1", role: "admin" })); }); }); test("admin creates a new
staff account", async ({ page }) => { await page.route("/api/v1/auth/admin/users*", async (route) => { if
(route.request().method() === "POST") { await route.fulfill({ status: 201, contentType: "application/json", body:
JSON.stringify({ success: true, data: { id: "new-1", status: "active" } }), }); } else { await route.fulfill({
status: 200, contentType: "application/json", body: JSON.stringify({ success: true, data: { users: [], total: 0 }
}), }); } }); const adminPage = new AdminUsersPage(page); await adminPage.goto(); await adminPage.createUser({
email: "newstaff@ejust.edu.eg", password: "StrongPass1!", displayName: "New Staff", role: "staff", }); await
expect(adminPage.userModal).not.toBeVisible(); }); });
```

E2E-06: Password Reset (FR06)

```
// tests/e2e/password-reset.spec.js import { test, expect } from "@playwright/test"; import { LoginPage } from
"./pages/LoginPage"; import { PasswordResetPage } from "./pages/PasswordResetPage"; test.describe("E2E: Password
Reset", () => { test("shows confirmation after submitting reset request", async ({ page }) => { await
page.route("/api/v1/auth/password-reset/request", async (route) => { await route.fulfill({ status: 202, contentType:
"application/json", body: JSON.stringify({ success: true, data: { message: "If this email is registered, a reset
link has been sent." }, }), }); }); const loginPage = new LoginPage(page); const resetPage = new
PasswordResetPage(page); await loginPage.goto(); await resetPage.openFromLogin(loginPage); await
resetPage.requestReset("ali@ejust.edu.eg"); await resetPage.waitForConfirmation(); await
expect(page.getByText("Check your email")).toBeVisible(); }); });
```

E2E-07: Order Cancellation Within Window (FR37)

```
// tests/e2e/cancel-order.spec.js import { test, expect } from "@playwright/test"; test.describe("E2E: Order
Cancellation", () => { test("student cancels order within 2-minute window", async ({ page }) => { await
page.goto("/menu"); await page.evaluate(() => { localStorage.setItem("jwt_token", "mock.jwt.token");
localStorage.setItem("user", JSON.stringify({ id: "u1", role: "student" })); }); // Mock cancel API await
page.route("/api/orders/*/cancel", async (route) => { await route.fulfill({ status: 200, contentType:
"application/json", body: JSON.stringify({ success: true, message: "Order cancelled successfully", refund: null, }),
}); }); // ■■ Given an order placed less than 2 minutes ago ■■ // ■■ When the student clicks Cancel ■■ // ■■ Then
the order status changes to CANCELLED ■■ }); });
```

Playwright Configuration

```
// frontend/playwright.config.js (existing) import { defineConfig, devices } from "@playwright/test"; export default
defineConfig({ testDir: "./tests", timeout: 30000, retries: 0, use: { baseURL: "http://localhost:5173", trace:
"on-first-retry", screenshot: "only-on-failure", }, projects: [ { name: "chromium", use: { ...devices["Desktop
Chrome"] } }, { name: "firefox", use: { ...devices["Desktop Firefox"] } }, { name: "mobile", use: {
...devices["Pixel 5"] } }, ], });
```

POM Coverage Matrix

Page Object	Gherkin Scenarios Covered	FRs

LoginPage	Login success, failure, lockout	FR01, FR03, FR08
MenuPage	Browse menu, add to cart	FR09, FR11
OrderPage	Place order, cancel, payment select	FR20, FR26, FR37
PasswordResetPage	Request reset, confirmation	FR06
AdminUsersPage	Create user, suspend, edit role	FR50, FR51